

Benchmarking / Performance Test Strategy

Benchmarking / Performance Test Strategy

Revision: 1 Last modified: 2026-06-26T12:00:00Z

Master technical specification — Volume 8 (Testing & QA), nano-detail document **benchmarking**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens ~~10-testing-acceptance-and-qa.md~~ §5.13 (BENCH + PERF + SCALE) into an implementation-ready bank for throughput, CPU-per-Gbps, handshakes/sec, p50/p95/p99 latency, and the Go-vs-Rust edge decision (G4). **Every figure here is a TARGET measured against a recorded baseline (§11.4.6) — a regression vs baseline is a finding, never a soft warning.** SPEC-ONLY: it describes harness, fixtures, evidence, gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — [OVERVIEW] = doc 10; [04_P0] = Phase-0 spike (bench.sh, G1/G2/G4); [SYNTHESIS] = synthesis (D5 edge language); [reconcile] = [../v03-control-plane/reconciliation-flow.md]; [ladder] = [../v02-data-plane/transport-selection-ladder.md]; [01-DP] = doc 01; [TM] = [../v05-security/threat-model.md]. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated.

Table of contents

- 0. Scope + the figure-is-a-target-not-a-claim rule
 - 1. The benchmark + SLO matrix
 - 2. Harness — bench.sh + criterion
 - 3. The baseline registry + regression rule
 - 4. Fixtures — real rig (§11.4.27)
 - 5. Captured evidence (§11.4.69 / .24)
 - 6. Determinism (§11.4.50)
 - 7. Acceptance gate
 - 8. The paired §1.1 mutation
 - 9. Test skeletons
 - Sources verified
-

0. Scope + the figure-is-a-target-not-a-claim rule

Performance is the test type most prone to bluff-by-optimism: it is tempting to *assert* "we do 80% of bare-link" without measuring. §11.4.6 forbids it — **every number in this bank is either a TARGET (a go/no-go bar) or a MEASURED RESULT (captured from a real run), never a design estimate stated as a result.** A measured figure that regresses below the recorded baseline is a **finding** (a release blocker), processed via §11.4.4 STOP + systematic-debug — not a soft warning the suite shrugs off.

Sub-type	What it measures	HelixVPN surface
BENCH	raw throughput, CPU-per-Gbps, handshakes/sec, framing latency	edge data plane (Rust/Go), WG core, MASQUE framing
PERF	p50/p95/p99 latency vs SLO budget; Go GC-tail	control-plane reconcile/enroll/revoke hot paths
SCALE	N agents holding streams; convergence + bounded memory	coordinator fan-out (partial MVP / full Phase 2)

The headline number is always the **measured** reduction/throughput, never the design estimate (§11.4.141 lesson applied to perf): the spec states the *target*, the run produces the *result*, and the gate compares result-to-target and result-to-baseline.

1. The benchmark + SLO matrix

Metric	Target (the bar)	Type	Gate / SLO	Baseline source
plain-UDP WG throughput	$\geq 80\%$ of bare-link	BENCH	G1 [04_P0]	iperf3 bare-link run (same rig)
MASQUE/QUIC throughput	$\geq 50\%$ of plain-UDP; survives 5% loss	BENCH	G2 [04_P0]	the G1 plain-UDP run
CPU-per-Gbps (Rust vs Go edge)	lower is better; decides G4	BENCH	G4 (edge language) [SYNTHESIS D5]	the other language's run
handshakes/sec	$\geq$ target (Phase-2 DDoS seam input)	BENCH	bench trend	prior tagged run
MASQUE framing latency	within budget	BENCH (criterion)	bench trend	prior criterion baseline
event→delta-on-wire p99	$< 1\text{ s}$	PERF	SLO1	prior tagged run

Metric	Target (the bar)	Type	Gate / SLO	Baseline source
enroll→first NetworkMap	< 2 s	PERF	SLO2	prior tagged run
revoke→edge enforcement p99	< 1 s	PERF	SLO3	prior tagged run
coordinator mem @ 10k streams	bounded / 24 h	SCALE	SLO4 (→ [memory.md])	prior soak
N-agent convergence p99	< 1 s	SCALE	partial MVP / full Phase 2	prior soak

The G4 row is special: it is a **decision input**, not a pass/fail — the CPU-per-Gbps + p99 + churn

- memory CSV for Rust vs Go decides `gates.G4.outcome` IN ('rust', 'go') [04_P0 §8].

2. Harness — bench.sh + criterion

Layer	Tool	What it produces
edge throughput / goodput	<code>bench.sh</code> driving the netns rig with <code>iperf3 -J</code>	per-iteration throughput + loss-goodput CSV
CPU-per-Gbps	<code>iperf3</code> throughput ÷ edge CPU-seconds (/proc or cgroup <code>cpuacct</code>)	the G4 decision CSV
micro-bench (MASQUE framing, WG encode/decode)	<code>criterion</code> (Rust)	statistically-rigorous per-op latency with confidence intervals
control-plane PERF p99	histogram metrics (<code>helix_reconcile_seconds</code>) + a timed driver	p50/p95/p99 per SLO
SCALE convergence	the coordinator agent-fuzzer holding N streams	convergence p99 + RSS series

```
# bench.sh – the G1/G2/G4 benchmark, deterministic over N=3
# (overview §5.13, [04_P0 §8])
THRPUT=$(iperf3 -J -c 10.10.0.20 -P "$CLIENTS" | jq
'.end.sum_received.bits_per_second')
GOODPUT_LOSS=$(with_netem 'loss 5% delay 40ms'
iperf3_goodput) # G2 / resilience
CPU_PER_GBPS=$(awk -v t="$THRPUT" -v c="$EDGE_CPU_SEC"
'BEGIN{print c/(t/1e9)}') # G4 input
```

```
printf '%s,%s,%s,%s,%s\n' "$EDGE_LANG" "$RUN" "$THRPUT"
"$GOODPUT_LOSS" "$CPU_PER_GBPS" \
>> qa-results/bench/edge_compare.csv      # the G4 decision CSV
(rust vs go)
```

The bank runs under `make bench` (overview §9: `bash bench.sh 3` — `N=3` deterministic), backgrounded (§11.4.89). The netns rig needs `CAP_NET_ADMIN` (scoped sudo exception); criterion and the control-plane PERF drivers run under `containers-booted infra` (§11.4.76, rootless §11.4.161).

3. The baseline registry + regression rule

§11.4.6 + §11.4.24: a number means nothing without a baseline to compare to. The bank maintains a **git-tracked baseline registry** — a TSV per metric per platform per edge-language, with the last-accepted value and its run context (artifact MD5, rig fingerprint, CLIENTS, date). Every run:

1. measures the metric `N=3` (overview §5.13) and computes min/max/mean/p95 (§11.4.24 discipline);
2. compares the run's mean (or p99 for latency) to (a) the **target** bar and (b) the **baseline**;
3. classifies: **PASS** (meets target AND $\geq$ baseline – tolerance), **REGRESSION** (below baseline beyond tolerance — a **finding**, §11.4.4 STOP), or **IMPROVEMENT** (above baseline — update the baseline in the same commit with the evidence).

regression rule (§11.4.6):

```
run.metric < baseline.metric - tolerance    => REGRESSION =>
finding (release blocker)
run.metric within tolerance of baseline    => PASS
run.metric > baseline.metric + tolerance   => IMPROVEMENT =>
ratchet the baseline (commit w/ evidence)
```

A regression is never silently accepted (no `--allow-perf-regression` escape) — it is a finding processed like any defect (systematic-debug per §11.4.102, fix or justify with evidence). The baseline is ratcheted *up* only with a captured-evidence run (§11.4.135-class: the baseline is a standing regression guard for performance).

4. Fixtures — real rig (§11.4.27)

Fixture	What	Why real
netns rig (client→gateway→connector LAN)	the real E2E reachability rig (overview §5.3)	throughput must cross a <i>real</i> tunnel, not a mocked socket
real helix-edge (Rust + Go builds)	both candidate edge artifacts for G4	CPU-per-Gbps must measure the <i>real</i> edge in each language
tc netem loss/delay profile		

Fixture	What	Why real
	a real impairment profile (5% loss, 40 ms delay)	G2 goodput-under-loss must use real netem, not a simulated penalty
baseline TSV registry	the git-tracked accepted-baseline values	the regression comparison needs a real prior result

The bare-link reference (the denominator of G1's "80% of bare-link") is itself a measured `iperf3` run on the same rig **without** the tunnel — a real number, captured the same cycle, never an assumed line rate (§11.4.6).

5. Captured evidence (§11.4.69 / .24)

Every PASS cites a CSV under `qa-results/bench/<run-id>/` with min/max/mean/p95 (§11.4.24). The §11.4.69 class is `network_throughput` (BENCH) / a counter-delta latency (PERF).

Test	Artifact	Asserts
BENCH-G1-THROUGHPUT	<code>g1_throughput.csv</code>	mean $\geq$ 80% of the captured bare-link run
BENCH-G2-MASQUE	<code>g2_masque.csv</code>	mean $\geq$ 50% of the G1 run; goodput survives 5% loss
BENCH-G4-EDGE-COMPARE	<code>edge_compare.csv</code>	the Rust-vs-Go CPU-per-Gbps/p99/mem decision matrix
PERF-SLO1-RECONCILE	<code>reconcile_p99.csv</code>	<code>helix_reconcile_seconds</code> p99 < 1 s
PERF-SLO2-ENROLL	<code>enroll_latency.csv</code>	<code>enroll</code> → <code>first-map</code> < 2 s
PERF-SLO3-REVOKE	<code>revoke_latency.csv</code>	<code>revoke</code> → <code>edge-enforcement</code> p99 < 1 s
SCALE-CONVERGE	<code>converge_p99.csv</code>	N-agent convergence p99 < 1 s

The CSV with the percentile distribution is the evidence (§11.4.107(13): thresholds calibrated on the project's own rig, not literature). A PERF PASS that cites only a single latency sample is a point-not-window bluff — the gate asserts a **percentile over a window** (p99), and §11.4.107(2)'s independent-counter-advance applies (goodput AND the WG transfer counter must both move, or it is a decoy path).

6. Determinism (§11.4.50)

`bench.sh` 3 runs each benchmark N=3 against the same artifact MD5 + same rig + same CLIENTS; the suite computes min/max/mean/p95 and asserts the spread is within a tolerance band (a benchmark whose runs vary by $> X\%$ is itself a non-

deterministic *measurement* and is auto-FAIL — the number is not trustworthy). The determinism evidence-hash is over the verdict tuple (`meets_target: bool`, `not_regressed_vs_baseline: bool`); all 3 runs MUST agree on the verdict. The raw throughput numbers vary run-to-run (that is physical), so the *verdict* (target met / not regressed), not the raw number, is the determinism key — the same discipline as the DDoS/chaos verdict-determinism. PERF p99 is taken over a large sample within each of the 3 runs so the percentile itself is stable.

7. Acceptance gate

Gate / SLO	Bar (TARGET)	Evidence	Phase	Regression handling
G1	throughput $\geq$ 80% bare-link	g1_throughput.csv	Phase 0	below baseline $\Rightarrow$ finding
G2	MASQUE $\geq$ 50% plain; survives 5% loss	g2_masque.csv	Phase 0	finding
G4	record CPU-per-Gbps decision (rust go)	edge_compare.csv	Phase 0 (decision)	n/a (decision)
SLO1	event $\rightarrow$ delta p99 < 1 s	reconcile_p99.csv	MVP	finding
SLO2	enroll $\rightarrow$ first-map < 2 s	enroll_latency.csv	MVP	finding
SLO3	revoke $\rightarrow$ edge p99 < 1 s	revoke_latency.csv	MVP	finding
SLO4	coordinator mem bounded @ 10k ($\rightarrow$ [memory.md])	coord_rss_24h.csv	MVP	finding
SCALE-CONVERGE	N-agent convergence p99 < 1 s	converge_p99.csv	partial MVP / full P2	finding

A PERF regression beyond budget is a **release blocker** (overview §5.13). G4 is a decision gate (records rust/go), not pass/fail. BENCH/PERF cells appear in the ledger for F-AUTHZ-REACH (G1), F-TRANSPORT-ESCALATE (G2), F-POLICY-RECONCILE (SLO1), F-REVOKE (SLO3) (overview §6.3, §7.2). These are not the §11.4.132 risk-ordered *head* (the security floor runs first) but a PERF regression on a hot path is still a blocker.

8. The paired §1.1 mutation

MUTATION (paired §1.1, gate CM-PERF-REGRESSION-IS-FINDING):
Inject a fixed 200 ms sleep into the reconcile hot path (or lower the recorded SL01 baseline so a regressed run would "pass").
EXPECTED (sleep variant): reconcile_p99.csv p99 blows past 1 s → PERF-SL01 FAILs → mutation caught.
EXPECTED (baseline variant): the regression-rule comparator no longer flags the regressed run → the baseline-tamper meta-test FAILs (the baseline must be a guard, not a movable goalpost).
RESTORE: remove the sleep / restore the baseline; re-run → GREEN.

A second mutation (CM-G1-BARELINK-DENOMINATOR) replaces the *measured* bare-link denominator with an assumed line-rate constant; expected: the "denominator must be a captured run, not a constant" check FAILs (defeats the bluff-by-optimism of asserting a percentage against an imaginary baseline, §11.4.6). A third (CM-BENCH-COUNTER-ADVANCE) reports goodput from iperf3 while the WG transfer counter is flat; expected: §11.4.107(2) independent-counter-advance check FAILs (a decoy/loopback path). All mutations restored, tree verified quiescent (§11.4.84) before commit.

9. Test skeletons

```
# rig/bench_g1.sh - BENCH-G1: throughput ≥ 80% of the MEASURED
#                       bare-link (never assumed)
set -euo pipefail
out="qa-results/bench/$(date +%s)"; mkdir -p "$out"
trap 'rig/netns_down.sh' EXIT
bare=$(measure_bare_link_iperf3) #
# REAL bare-link run, same rig (the denominator)
tun=$(ab_run_n_times "g1" 3 'measure_tunnel_iperf3' )
# N=3 deterministic; mean throughput
wg_rx_advanced=$(wg_transfer_counter_advanced)
# §11.4.107(2) independent counter
pct=$(awk -v t="$tun" -v b="$bare" 'BEGIN{print 100*t/b}')
regressed=$(compare_to_baseline g1_throughput "$tun")
# §3 regression rule
[ "$(awk -v p="$pct" 'BEGIN{print (p>=80)}')" = 1 ] && [
    "$wg_rx_advanced" = 1 ] && [ "$regressed" = no ] \
    && ab_pass_with_evidence "G1: ${pct}% of bare-link (>=80%), no
    regression" "$out/g1_throughput.csv" \
    || ab_fail "G1: ${pct}% (target 80%);
    wg_advanced=$wg_rx_advanced regressed=$regressed"

# rig/bench_g4.sh - BENCH-G4: Rust-vs-Go edge CPU-per-Gbps
#                       decision CSV [04_P0 §8, SYNTHESIS D5]
for lang in rust go; do
    deploy_edge "$lang"
```

```

EDGE_LANG="$lang" RUN=1 bash bench.sh 3
    # appends to edge_compare.csv (N=3)
done
decision=$(decide_edge_language qa-results/bench/
    edge_compare.csv) # lower CPU-per-Gbps + p99 + churn +
    mem
record_gate_outcome G4 "$decision"
    # gates.G4.outcome IN ('rust','go') with evidence_path
ab_pass_with_evidence "G4 decision = $decision" "qa-results/bench/
    edge_compare.csv"

# rig/perf_slo3_revoke.sh – PERF-SLO3: revoke-edge enforcement p99
    < 1 s (overview §7.2)
set -euo pipefail
out="qa-results/bench/$(date +%s)_revoke"; mkdir -p "$out"
for i in $(seq 1 200); do
    t0=$(now_ms); issue_revoke "dev-$i"; wait_for_edge_peer_removed
        "dev-$i"; t1=$(now_ms)
    echo "$((t1 - t0))" >> "$out/revoke_latency.csv"
done
p99=$(percentile "$out/revoke_latency.csv" 99)
[ "$p99" -lt 1000 ] && [ "$(compare_to_baseline revoke_p99
    "$p99")" = no_regression ] \
    && ab_pass_with_evidence "SLO3 revoke p99=${p99}ms (<1s), no
        regression" "$out/revoke_latency.csv" \
    || ab_fail "SLO3 revoke p99=${p99}ms – finding (§11.4.4 STOP)"

```

Honest boundary (§11.4.6). Every target figure in §1 (80% bare-link, 50% MASQUE, < 1 s SLOs) is a **TARGET**, not a measured result, until the rig produces the CSV — they are **UNVERIFIED** as HelixVPN results and become facts only when bench.sh/the PERF drivers run on the real rig and the captured CSV is attached. The G4 edge-language outcome is **UNVERIFIED** until the edge_compare.csv exists (the spec records it as a decision to be made from the CSV, never a pre-decided result). The SCALE convergence p99 at the MVP agent count is partial; full N-scale is a Phase-2 SCALE concern ([TM T-COORD-D-1] flags the 10k-stream p99 as a soak number not yet a result). The bare-link reference rate depends on the rig's NIC/CPU and is captured per-rig, never assumed.

Sources verified

- [OVERVIEW] [../10-testing-acceptance-and-qa.md] — §5.13 (BENCH+PERF+SCALE strategy + bench.sh skeleton), §3.2 (metamorphic 2×-clients relation), §6.3 (BENCH/PERF ledger cells), §7.1 (G1/G2/G4), §7.2 (SLO1–SLO4), §8 (determinism N=3), §9 (make bench). (Read 2026-06-26.)
- [04_P0] 04_VPN_CLD/HelixVPN-Phase0-Spike.md — §8 bench.sh, G1 ≥80% bare-link, G2 ≥50% MASQUE, G4 Rust-vs-Go CPU-per-Gbps decision. (Cited via overview.)
- [SYNTHESIS] v09-research/_SYNTHESIS.md — D5 edge-language decision input. (Cited via overview.)
- [reconcile] [../v03-control-plane/reconciliation-flow.md] — helix_reconcile_seconds histogram, event→enforced p99 < 1 s budget. (Read 2026-06-26.)

- [ladder] [../v02-data-plane/transport-selection-ladder.md] — MASQUE rung (G2 throughput surface). (Read 2026-06-26.)
- [01-DP] final/01-data-plane.md — WG core throughput, MASQUE framing; [TM] [../v05-security/threat-model.md] T-C00RD-D-1 (10k-stream p99 **UNVERIFIED**). (Cited via overview.)
- Constitution: §11.4.169, §11.4.6 (TARGET-not-claim / UNVERIFIED / regression-is-finding), §11.4.24 (min/max/mean/p95 resource stats), §11.4.27 (real rig), §11.4.69 (throughput evidence class), §11.4.107(2)/(13) (independent counter-advance + calibrated thresholds), §11.4.50 (determinism N=3), §11.4.135 (baseline as regression guard), §11.4.4 (STOP on regression), §11.4.102 (systematic-debug), §11.4.84 (quiescence), §1.1 (paired mutation).